

High-Performance Computing Architecture Exploration with Stage-Enhanced Bayesian Optimization

Vincent Fu^{1,2}, Mohamed Benazouz¹, Lilia Zaourar¹, Alix Munier-Kordon²

¹Université Paris-Saclay, CEA, List, F-91120 Palaiseau, France

²Sorbonne Université, CNRS, LIP6, F-75005 Paris, France

{vincent.fu, mohamed.benazouz, lilia.zaourar}@cea.fr, alix.munier@lip6.fr

Abstract—The emergence of new applications in high-performance computing is driving the need for more efficient computing machines. As supercomputer architectures become increasingly complex, the combinatorial explosion of design spaces and the time-consuming nature of design simulations lead to challenging design space exploration problems. This work introduces an automated search framework to achieve power-performance-area efficient Arm Neoverse V1 processor designs. Based on multi-objective Bayesian optimization, we propose a new exploration algorithm named SEBO by enhancing the three main stages of the optimization. Experimental results show that SEBO can not only compete with the top state-of-the-art baseline algorithms, but also outperforms them in terms of the quality and diversity of the returned Pareto-optimal designs.

Index Terms—High-performance computing, design space exploration, Bayesian optimization, ensemble learning

I. INTRODUCTION

High-Performance Computing (HPC) is at the heart of recent advances in large-scale processing and modeling problems. From weather forecasting and physics simulations to machine learning and data analysis, these applications are becoming more and more computational intensive, leading to a sophistication of supercomputer architectures within the Multi-Processor System-on-Chip (MPSoC) design paradigm.

To ensure their efficiency before chip manufacturing, heavy simulation-based design is the state-of-the-art approach [5]. By running target applications on virtual prototypes, it gives designers access to key performance indicators (latency, power, etc.) of simulated architectures. The challenge is to figure out efficient architectural configurations by tuning through simulations, various hardware parameters such as core count, cache size, Network-on-Chip (NoC) topology, etc. However, due to the combinatorial explosion of design spaces and the time-consuming nature of design simulations, engineers must adopt a co-design approach by carefully selecting which architectural configurations to prioritize and for which target applications. Analytical formulations [32], [34] have been proposed as a simulation-substitute method for calculating performance metrics, but it requires confidential data from industry to correctly parameterize formulas and their coefficients. As a result, the choice of the final architectural configuration often relies on expert knowledge gained from engineering experience.

On the other hand, Design Space Exploration (DSE) [27] methodologies have shown impressive results for reaching more efficient designs [1], [22] than expert-made ones.

Machine learning and operations research techniques provide powerful exploration algorithms to overcome high-dimensional multi-objective optimization difficulties. In this work, an automated design space exploration framework is proposed based on an advanced state-of-the-art Bayesian optimization algorithm for efficient 7nm Arm Neoverse V1 processor designs. Our contributions are as follows:

- The automated design space exploration framework uses VPSim [7], a SystemC transaction-level modeling (TLM) simulator, and a 7nm-supported McPAT [21], [38] modeler to compute Power-Performance-Area (PPA) estimations with a good simulation speed-accuracy trade-off.
- We propose a new exploration algorithm named SEBO, for Stage-Enhanced Bayesian Optimization, featuring a two-step Hammersley-transductive experimental design [37] model initialization, a trust region-based [12] ensemble surrogate modeling, and a batch acquisition function with parallel simulations. Each stage of the optimization is optimized to unveil an end-to-end efficient framework.
- The experimental results were validated for various standard HPC applications on large Arm Neoverse V1 architectural configurations with up to 128 cores. SEBO has been compared with the top state-of-the-art DSE baselines [1], [4], [25] and shows excellent performance in terms of the quality and diversity of the returned Pareto-optimal designs for a reasonable running time.

The paper is structured as follows. Section II discusses the state of the art of exploration algorithms. Section III depicts our proposed framework, followed by experimental results and observations in Section IV. Finally, Section V ends with conclusion and perspectives.

II. RELATED WORK

A. Model-free and model-based approaches

Exploration algorithms can be divided into two main classes: model-free and model-based approaches [27]. Understanding these is key to strengthening exploration by selecting the most adapted one to each workflow's specificities.

Early works focused on model-free approaches with algorithmic improvements. Platune [19], a platform-tuning environment, used Pareto local search with parameter clustering for efficient MIPS R3000 processor SoC design. By building a parameter dependency model, Platune enables exploration at

scale through design space reduction. The high-level synthesis DSE tool Cwbexplorer [29] has also adopted a clustering approach with simulated annealing. Despite the benefits of clustering, Palesi and Givargis [26] found that it may be insufficient to significantly reduce the design space and further improved Platune's scalability using the genetic algorithm SPEA2. However, since the model-free algorithms mentioned above heavily rely on heuristic functions, the exploration outcomes may be driven by some degree of intrinsic randomness. That means model-free algorithms are only efficient for high evaluation budgets. The greater the number of evaluations, the more quantitative information the randomness provides.

While evaluating a single configuration in Platune takes on average 1.5s, other frameworks like the VLSI flow of RISC-V Berkeley Out-of-Order Machine (BOOM) [1] can require 6 to 14 hours. In such cases, the randomness of model-free algorithms becomes undesirable, making sample-efficiency a critical feature to consider. In contrast, model-based approaches leverage machine learning models to capture the relationship between design parameters and objective functions. A typical example of that is reinforcement learning. Beltrame *et al.* [3] for instance, translated the domain knowledge of an MPSoC into a discrete-space Markov decision process and found the optimal exploration policy with value iteration. Lin *et al.* [22] went further using Monte-Carlo tree search to learn a deep policy network for efficient routerless NoC design. However, deriving optimal policies through reinforcement learning techniques is naturally sample-expensive, leading them to be more suitable for learning optimal control in applications like robotics or autonomous agents rather than for expensive simulation-based objective optimization.

B. Bayesian optimization: a sample-efficient methodology

As a similar learning-based alternative, Bayesian optimization [18] offers a more sample-efficient strategy. It sequentially employs two key components: the surrogate model and the acquisition function. After each design evaluation, the results are fed into the surrogate model to improve the objective function approximation. The acquisition function then leverages the surrogate model as a fully information-aware decision making tool to optimally suggest the next design to evaluate and so on. The sample-efficiency is therefore guaranteed by the surrogate model's ability to approximate the objective function precisely.

Among Bayesian optimization-based DSE frameworks, HyperMapper [25] proposed a random forest surrogate model for each objective function and combined with the expected improvement acquisition function using a randomized Chebyshev aggregation. Although random forests offer strong non-linear regression for discrete design spaces, their ability to model uncertainty as accurately as Gaussian processes, commonly used in Bayesian optimization, is debatable. In contrast, BOOM-Explorer [1] takes advantage of the latter and increases their modeling power through deep kernel learning. Moreover, since surrogate models need to be initialized, Bai *et al.* also implemented a clusterized transductive experimental design [37] as initial sampling for an informative warm start.

Despite the promising results of Bayesian optimization in DSE problems, most advanced Bayesian optimization techniques have been studied beyond hardware design. In the NeurIPS 2020 black-box optimization challenge [31], participants were asked to implement an efficient algorithm that outperforms the strongest baseline TuRBO [16], a new state-of-the-art Bayesian optimization algorithm, for various machine learning hyperparameter tuning tasks. The main conclusions of the challenge suggest that, first, Bayesian optimization was consistently effective. Not only did less-efficient Bayesian optimization-based baselines outperform genetic algorithms, but also all of the top proposed methods were Bayesian optimization techniques. Secondly, to outperform TuRBO, top participants naturally employed its main feature: using trust regions for local Bayesian optimization and incremented, on top of that, multiple surrogate and acquisition models as ensemble learning. Considering the no free lunch theorems [35], leveraging multiple surrogate models can further improve the approximation of objective functions.

To illustrate how the fit of surrogate models impacts exploration, Dreczkowski *et al.* [15] conducted a comparative study on discrete kernel-based Gaussian processes for combinatorial Bayesian optimization. They found that some kernels, such as the string subsequence [20], the overlap, and the Hamming embedding via dictionary [13], provide better model fitting than others across various benchmarking tasks. Furthermore, they also observed that trust region-based methods consistently outperform the non-trust region ones, confirming the efficiency of local modeling for any problem instances.

Assuming hardware design optimization deals with discrete parameters, leveraging trust regions and ensemble learning with efficient discrete space-tailored surrogate models is key to achieving a state-of-the-art Bayesian optimization-based DSE.

III. AUTOMATED EXPLORATION FRAMEWORK

Fig. 1 shows our proposed framework and provides an overview of how an automated model-based DSE works. By alternating between design suggestions and simulation feedback, the exploration model progressively accumulates relevant information, which further enhances this alternating mechanism, driving the convergence toward efficient designs. Known as active learning, the way in which the model makes suggestions and updates itself with new information is central to exploration performance.

A. Multi-objective optimization problem

In hardware design, the quality of an architecture is measured through different key performance indicators, such as energy consumption, latency, and area. Since the latter are negatively correlated, the ideal design, which is the one that would have the best value for each metric at the same time, is in reality, not feasible. Trying to optimize one objective comes at the cost of degrading another. While in single-objective optimization, the optimality is trivially deduced from the total order of real numbers, searching optimal design in multi-

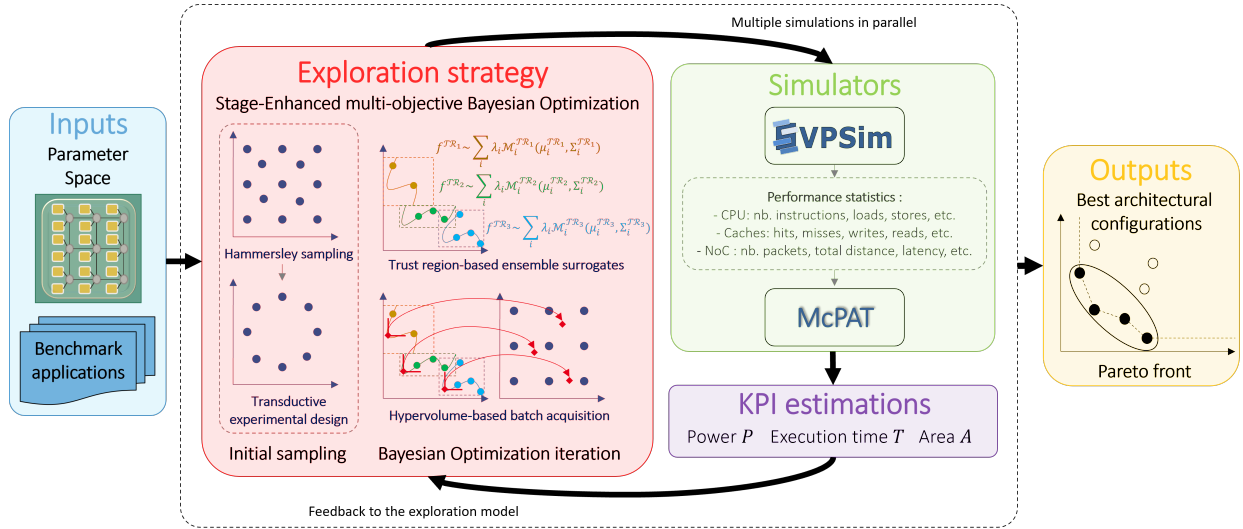


Fig. 1: Overview of proposed automated design space exploration framework

objective optimization comes down to making objective trade-offs defined by the Pareto optimality.

Definition 1 (Pareto optimality). *Let $\mathcal{X}$ be the design space and $M \geq 2$. Given a M -dimensional multi-objective function $F(\mathbf{x}) = [f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_M(\mathbf{x})]^\top$, a solution $\mathbf{x} \in \mathcal{X}$ is said to be dominated by $\mathbf{y} \in \mathcal{X}$, denoted as $\mathbf{x} \preceq \mathbf{y}$, in minimization (resp. maximization) problem if and only if:*

$$\begin{aligned} \forall p \in [1, M], \quad f_p(\mathbf{y}) \leq f_p(\mathbf{x}) \quad (\text{resp. } f_p(\mathbf{y}) \geq f_p(\mathbf{x})) \\ \exists q \in [1, M], \quad f_q(\mathbf{y}) < f_q(\mathbf{x}) \quad (\text{resp. } f_q(\mathbf{y}) > f_q(\mathbf{x})) \end{aligned} \quad (1)$$

A solution $\mathbf{x}$ is Pareto-optimal if and only if no other solution dominates it (i.e. $\nexists \mathbf{y} \in \mathcal{X}, \mathbf{x} \preceq \mathbf{y}$).

Since different objective trade-offs are possible (e.g., a design can be more latency efficient at the cost of higher energy consumption, or *vice versa*), the goal of multi-objective optimization is to return all feasible Pareto-optimal solutions.

Therefore, achieving PPA-efficient hardware designs involves multi-objective optimization. Given a design space $\mathcal{X}$, we want to retrieve the Pareto set $\mathcal{X}_P$ by simultaneously minimizing the energy consumption e , the execution time t and the area a given a target application γ :

$$\min_{\mathbf{x} \in \mathcal{X}} F(\mathbf{x}) = [e(\mathbf{x}, \gamma), t(\mathbf{x}, \gamma), a(\mathbf{x})]^\top \quad (2)$$

$$\mathcal{X}_P = \{\mathbf{x} \in \mathcal{X} \mid \nexists \mathbf{y} \in \mathcal{X}, \mathbf{x} \preceq \mathbf{y}\} \quad (3)$$

In our problem instance, a design $\mathbf{x}$ is in the design space $\mathcal{X}$ if its architectural parameters are among candidate values defined in TABLE I and satisfy hardware constraints.

As analytical formulations are not exploited here, the multi-objective values $F(\mathbf{x})$ are obtained through design simulations using various electronic design automation tools. In particular, we leverage VPSim and an enhanced version of McPAT [38] to build our simulation flow for Arm processor design.

B. VPSim simulator and McPAT modeler

VPSim [7] is a SystemC TLM-based virtual prototyping tool dedicated to hardware-software co-design. Based on the

TABLE I: System-level design space of Arm Neoverse V1

Architectural parameters	Candidate values
Number of cores n_c	2, 4, 8, 16, 32, 64, 128
Number of cores per cluster n_q	1, 2, 4, 8, 16
L1 data/instruction cache size (KB) s_1	16, 32, 64, 128
L2 cache size (KB) s_2	256, 512, 1024, 2048
L3 cache size (KB) s_3	512, 1024, 2048, 4096, 8192
Cache block size (B) b	64, 128
L1 cache associativity a_1	2, 4
L2 cache associativity a_2	4, 8, 16
L3 cache associativity a_3	4, 8, 16, 32
Mesh NoC XY dimensions d_X, d_Y	1, 2, 4, 8, 16

emulation platform QEMU [2] augmented with fast memory hierarchy models, VPSim enables running software applications on simulated architectures for various processor models including Arm cores. With a good balance between simulation speed and accuracy, VPSim not only estimates application execution time but also provides performance statistics such as the number of instructions, cache reads/writes, hits/misses, etc. However, since it computes only one of the objectives, we use McPAT to handle the remaining objective computations.

McPAT [21] is a power and area modeling framework for many-core architectures. Built on top of CACTI [24], McPAT follows the same analytical approach to extend power and area evaluations for between 90nm to 22nm technology node-based MPSoC. Since McPAT needs to be combined with a performance simulator to function, the coupling of McPAT and VPSim is perfectly matched to form a complete PPA evaluation flow.

Furthermore, to support 7nm design, we draw inspiration from McPAT-Calib [38] by integrating 7nm FinFET technology [8] scaling parameters into McPAT's analytical models. Whereas McPAT-Calib leveraged on top of that, already simulated architecture data to train McPAT's models for better accuracy, we assume here to be design-agnostic, and to not use any prior simulation information before the start of DSE.

To automate DSE efficiently, we propose our own exploration algorithm named SEBO, based on improvements at

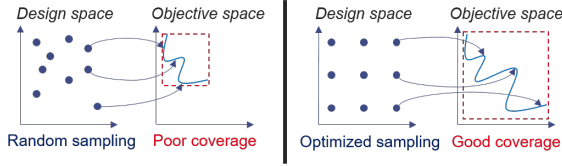


Fig. 2: Example of efficient sampling

three main stages of Bayesian optimization.

C. Stage-enhanced multi-objective Bayesian optimization

SEBO is a model-based algorithm based on Bayesian optimization principle [18] for multi-objective optimization. As SEBO employs surrogate models, they must first be initialized by training on an initial set of solutions with their corresponding labels (their objective values). In our problem, it means choosing initial architectural configurations and simulating them. This can be considered as the first stage of DSE.

A common method for selecting initial solutions is random sampling. Despite its simplicity, the stochasticity of random sampling can lead to a poor quality of initial solution set, ultimately biasing surrogate models by feeding them with undiversified information. Fig. 2 illustrates this impact.

In order to achieve an informative model initialization, we introduce the two-step Hammersley-transductive experimental design sampling as our first improvement. To understand the logic behind this, let's consider the ridge regression problem.

Definition 2 (Ridge regression). *Given n training samples $X = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n]^\top$, $\mathbf{y} = [y_1, y_2, \dots, y_n]^\top$ where $(\mathbf{x}_i, y_i) \in \mathbb{R}^N \times \mathbb{R}$ for $i = 1, \dots, n$, small $\lambda > 0$ and $\|\cdot\|$ the Euclidean norm, Ridge regression consists of solving:*

$$\hat{\mathbf{w}} = \arg \min_{\mathbf{w} \in \mathbb{R}^N} \|\mathbf{y} - X\mathbf{w}\|^2 + \lambda \|\mathbf{w}\|^2 \quad (4)$$

Trivially, $\hat{\mathbf{w}} = (X^\top X + \lambda I)^{-1} X^\top \mathbf{y}$ with identity matrix I . Considering regression is not exact due to the potential influence of unseen factors, the labels $\mathbf{y}$ are commonly modeled as $y_i = \mathbf{w}^\top \mathbf{x}_i + \epsilon_i$ where $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$ for $i = 1, \dots, n$. Then, the estimation error $\hat{\mathbf{w}} - \mathbf{w}$ has (with the zero vector denoted as $\mathbf{0}$ and $\epsilon = [\epsilon_1, \epsilon_2, \dots, \epsilon_n]^\top$):

$$\begin{aligned} \mathbb{E}[\hat{\mathbf{w}} - \mathbf{w}] &= (X^\top X + \lambda I)^{-1} X^\top \mathbb{E}[\epsilon] = \mathbf{0} \\ \text{Var}[\hat{\mathbf{w}} - \mathbf{w}] &= \text{Var}\left[(X^\top X + \lambda I)^{-1} X^\top \epsilon\right] \\ &= \sigma^2 (X^\top X + \lambda I)^{-1} X^\top X (X^\top X + \lambda I)^{-1} \end{aligned} \quad (5)$$

If the regression is exact, $\text{Var}[\hat{\mathbf{w}} - \mathbf{w}]$ should be equal to $\mathbf{0}$. $\text{Var}[\hat{\mathbf{w}} - \mathbf{w}]$ is therefore the metric which quantifies the amount of information that regression models cannot capture. Suppose we can now select our n training samples from the design space $\mathcal{X}$. Let V be the matrix composed of all designs $\mathbf{v} \in \mathcal{X}$. Selecting the most informative samples (for linear models) is equivalent to “minimize”:

$$\text{Var}[V\hat{\mathbf{w}} - V\mathbf{w}] = V \text{Var}[\hat{\mathbf{w}} - \mathbf{w}] V^\top = \sigma^2 C C^\top \quad (6)$$

where $C = V X^\top (X X^\top + \lambda I)^{-1}$ with the push-through identity. As we can notice, “minimizing” this variance doesn't

require any labels $\mathbf{y}$. This means we can select the most informative design solutions for our model initialization without simulating them. The kernel trick [30] is then applied within the regression framework to handle non-linear models, hence, defining the transductive experimental design problem (note that our formulation differs from the original paper [37]).

Definition 3 (Transductive experimental design). *Let $\mathcal{H}$ be a reproducing kernel Hilbert space with a kernel function $k(\mathbf{u}, \mathbf{z}) = \langle \phi(\mathbf{u}), \phi(\mathbf{z}) \rangle_{\mathcal{H}}$ where $\mathbf{u}, \mathbf{z} \in \mathbb{R}^N$ and $\phi: \mathbb{R}^N \rightarrow \mathcal{H}$ is a feature mapping. Supposing $\|\cdot\|_F$ is the Frobenius norm and $f \in \mathcal{H}$ has the form $f(\mathbf{x}) = \mathbf{w}^\top \phi(\mathbf{x})$, the optimal transductive experimental design is solution of, with matrix notation $K_{VX} = (k(\mathbf{v}_i, \mathbf{x}_j))_{1 \leq i \leq s, 1 \leq j \leq n}$:*

$$\begin{aligned} \min_X & \|K_{VX}(K_{XX} + \lambda I)^{-1}\|_F^2 \\ \text{s.t.} & \{\mathbf{x}_i\}_{i=1}^n \subseteq \{\mathbf{v}_i\}_{i=1}^s \end{aligned} \quad (7)$$

Since solving transductive experimental design over the entire design space is NP-hard due to combinatorial explosion, we leverage Hammersley sampling [9], [36] to build a low-discrepancy representative subset of the design space for V . We then solve transductive experimental design on it, using a scalable genetic algorithm. While BOOM-Explorer [1] relies on architecture-specific prior knowledge for pre-sampling its transductive experimental design, Hammersley sampling, by contrast, doesn't require any prior information about interactions between design space variables to sample, making it adapted for all DSE problem instances. Thus, our two-step Hammersley-transductive experimental design sampling forms the initial set of designs to be explored.

Given this first stage improvement, our second involves proposing accurate and expressive surrogate models. An efficient approach to achieve that is to employ Gaussian processes (GP) [28] for each objective. GP are non-parametric models that can be viewed as multivariate normal distributions over functions. In other words, $f \sim \mathcal{GP}(m, k)$ if for any finite collection of function values $\mathbf{f} = [f(\mathbf{x}_1), f(\mathbf{x}_2), \dots, f(\mathbf{x}_n)]^\top$:

$$\mathbf{f} \sim \mathcal{N}\left([m(\mathbf{x}_1), \dots, m(\mathbf{x}_n)]^\top, K_{XX}\right) \quad (8)$$

where m and k are the mean and variance functions. This definition implies that any $f(\mathbf{x}_i)$ is modeled by a Gaussian distribution, where the density function represents the plausibility of $f(\mathbf{x}_i)$ to be equal to the corresponding value from the support. By setting m and k correctly, these plausibilities which characterize our beliefs about the objective space in relation to the design space, can be parametrized, enabling machine learning process. Assuming simulations may be noisy, let $m(\mathbf{x})$ be associated with its observed value y as the empirical mean and k be a kernel function with hyperparameters θ , the objective values of an unexplored design $\mathbf{x}_* \in \mathcal{X}$ can be predicted by computing the posterior probability distribution $f(\mathbf{x}_*) | \mathbf{x}_*, X, \mathbf{y}$ using Bayes' rule with the joint distribution:

$$\begin{bmatrix} f(\mathbf{x}_*) \\ \mathbf{f} \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} m(\mathbf{x}_*) \\ \mathbf{y} \end{bmatrix}, \begin{bmatrix} k_\theta(\mathbf{x}_*, \mathbf{x}_*) & K_{\mathbf{x}_* X | \theta} \\ K_{X \mathbf{x}_* | \theta} & K_{XX | \theta} + \sigma^2 I \end{bmatrix}\right) \quad (9)$$

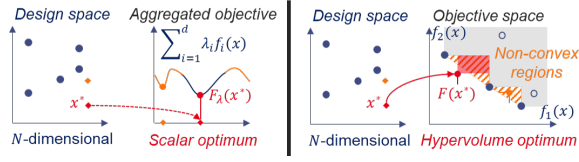


Fig. 3: Omission of non-supported Pareto-optimal solutions

where θ is tuned by maximizing the marginal likelihood. As their expressiveness depends on the kernel function, the choice of the latter becomes critical.

To mitigate the risk of choosing an unsuitable kernel, we bring ensemble learning by selecting the most appropriate kernel from the power set of different linear combinations of discrete space-tailored kernels at each iteration of Bayesian optimization and more importantly, for each objective function. In particular, we leverage the overlap k^O , the transformed overlap k^{TO} [33] and the Hamming embedding via dictionary k^{HED} as our base kernels to construct our ensemble $\mathcal{K}$:

$$\mathcal{K} = \left\{ \sum_{k \in \mathcal{E}} k \mid \mathcal{E} \in \mathcal{P}(\{k^O, k^{TO}, k^{HED}\}) \setminus \{\emptyset\} \right\} \quad (10)$$

$$k^O(\mathbf{u}, \mathbf{v}) = \frac{\sigma}{N} \sum_{i=1}^N \ell_i \delta(\mathbf{u}_i, \mathbf{v}_i) \quad k^{TO} = \exp\left(\frac{k^O}{\sigma}\right) \quad (11)$$

$$k^{HED}(\mathbf{u}, \mathbf{v}) = k_M^{5/2}(\phi_A(\mathbf{u}), \phi_A(\mathbf{v})) \quad (12)$$

where δ is the Kronecker delta, $k_M^{5/2}$ the Matérn 5/2 kernel, ϕ_A the embedding based on the dictionary A [13], ℓ_i are the lengthscales and the hyperparameters $\theta = (\sigma, \ell_1, \dots, \ell_N)$. Given a random test set $\mathcal{T}$ from already simulated designs and the density functions φ , the optimal kernel $k_\theta \in \mathcal{K}$ is selected by maximizing the sum of log predictive densities through leave-one-out cross validation:

$$k_\theta = \arg \max_{k \in \mathcal{K}} \sum_{(\mathbf{x}_*, \mathbf{y}_*) \in \mathcal{T}} \log(\varphi_{f(\mathbf{x}_*) | X, \mathbf{y} \setminus \{(\mathbf{x}_*, \mathbf{y}_*)\}}(\mathbf{y}_*)) \quad (13)$$

Although ensemble learning can improve model accuracy, it incurs a higher learning cost. To counter that, we leverage local Bayesian optimization by dividing the search space into trust regions [12], [16] to form several localized surrogate models trained on smaller datasets for faster multi-ensemble learning. Moreover, since each surrogate model is independent, we take advantage of training parallelization to accelerate our algorithm even further. For the trust region setup policy, we adopt the same policy from the original paper MORBO [12].

Let's now focus on our third stage improvement, which concerns sampling designs to be simulated using the acquisition function. Although there are popular ones [18] such as expected improvement and entropy search, all of them are single surrogate-based, making them unpractical for multi-objective Bayesian optimization. A common approach to break it down is to employ scalarization methods by aggregating all surrogate models into a single one. Nevertheless, optimizing an aggregated function such as a weighted linear aggregation can omit non-supported Pareto-optimal solutions in non-convex search space regions as illustrated in orange in Fig. 3.

To address this, we instead preserve the multidimensionality of multi-objective functions, and exploit hypervolume-based acquisition functions [10]. In particular, we leverage the noisy expected hypervolume improvement acquisition function [11] to even account the uncertainty in the Pareto front.

Definition 4 (Noisy expected hypervolume improvement). Given $\mathbf{r} \in \mathbb{R}^M$ a reference point, $\mathcal{D} = \{\mathbf{x}_i, \mathbf{y}_i\}_{i=1}^n$ the current set of observations, $\mathcal{Y}_P^{\mathcal{D}}$ the Pareto front of $\mathcal{D}$ in objective space, noisy expected hypervolume improvement is defined as:

$$\alpha_{\text{NEHVI}}(\mathbf{x}) = \int \alpha_{\text{EHVI}}(\mathbf{x} | \mathcal{Y}_P^{\mathcal{D}}) p(f | \mathcal{D}) df \quad (14)$$

where α_{EHVI} is the expected hypervolume improvement [10] and $p(f | \mathcal{D})$ the joint distribution of the Gaussian process f on $\mathcal{D}$.

Since (14) is analytically intractable, Daulton *et al.* proposed Monte-Carlo integration to approximate it. Maximizing the acquisition function therefore provides the most promising design according to the joint surrogate model. As the acquisition function can only be computed pointwisely, we use multi-start local search to determine our next design to evaluate. We perform this for each trust region-based joint model, all in parallel, as a batch acquisition to support multi-simulations.

With all the aforementioned features, SEBO is set for efficient automated DSE and can be interfaced with any electronic design automation tools. Its algorithm is outlined below.

Algorithm 1 SEBO($\mathcal{X}, s, n, n_{\text{TR}}$)

- 1 $V \leftarrow$ sample s Hammersley designs in $\mathcal{X}$ [36]
 - 2 $X \leftarrow$ solve *transductiveExperimentalDesign*(V, n) eq. (7)
 - 3 $Y \leftarrow$ *simulate*(X) **in parallel**
 - 4 Initialize trust regions using procedure [12] Section 3.2
 - 5 **while** simulation budget not exhausted **do**
 - 6 **for** trust region $\mathcal{TR}_i \in \llbracket 1, n_{\text{TR}} \rrbracket$ **do in parallel**
 - 7 $\mathcal{T}_i \leftarrow$ build random test set in $\mathcal{TR}_i$
 - 8 **for** objective $j \in \llbracket 1, M \rrbracket$ **do in parallel**
 - 9 $k_\theta \leftarrow$ find best kernel with $\mathcal{T} = \mathcal{T}_i$ eq. (13)
 - 10 Fit $f_j^{\mathcal{TR}_i} \sim \mathcal{GP}(m, k_\theta)$
 - 11 $\mathbf{x}_i \leftarrow \arg \max_{\mathbf{x} \in \mathcal{X} \cap \mathcal{TR}_i} \alpha_{\text{NEHVI}}(\mathbf{x})$ eq. (14)
 - 12 $\mathbf{y}_i \leftarrow \text{simulate}(\mathbf{x}_i)$, $X \leftarrow X \cup \{\mathbf{x}_i\}$, $Y \leftarrow Y \cup \{\mathbf{y}_i\}$
 - 13 Update trust regions with MORBO [12] lines 7-19
 - 14 **return** Pareto-optimal designs $\tilde{\mathcal{X}}_P$ and image values $\tilde{\mathcal{Y}}_P$
-

IV. EXPERIMENTAL RESULTS

To evaluate each aspect of candidate configuration efficiency, we chose STREAM [23], DGEMM [14], and waLBerla [17] as commonly used HPC benchmark applications. These benchmarks respectively target memory bandwidth, matrix computation, and numerical simulation performances.

We implemented SEBO, HyperMapper [25], and BOOM-Explorer [1] and interfaced them with our simulation flow to compare them on the same basis, on our benchmark applications. We also bring for the first time, a comparison with Hexaly Optimizer 13.0 (formerly known as LocalSolver) [4], a state-of-the-art commercial solver that provides simulation

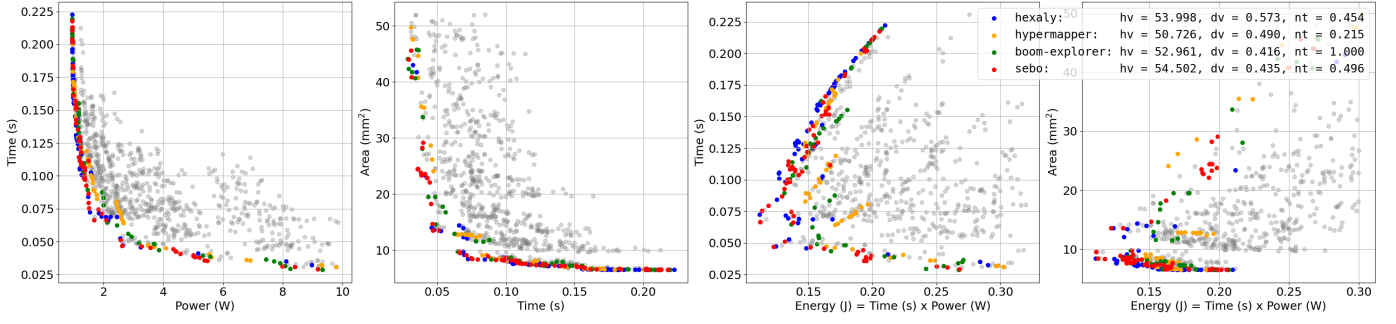


Fig. 4: Pareto fronts in bi-objective spaces with the hypervolume h_v , diversity d_v and normalized running time nt indicators

TABLE II: Key Pareto-optimal designs

<>-efficient	STREAM*			
	Design parameters [†]	Energy	Time	Area [‡]
<Energy>	[16, 16, 128, 2048, 4096, 64, 2, 16, 32, 1, 1]	0.960	0.135	36.24
<Time>	[16, 1, 16, 2048, 8192, 128, 4, 4, 4, 4, 4]	1.246	0.094	78.46
<Area>	[16, 16, 16, 256, 512, 64, 2, 8, 16, 1, 1]	2.384	0.427	28.92
<Trade-off> [‡]	[16, 16, 128, 2048, 4096, 64, 2, 16, 32, 1, 1]	0.960	0.135	36.24
<>-efficient	DGEMM			
	Design parameters [†]	Energy	Time	Area [‡]
<Energy>	[2, 2, 64, 2048, 8192, 128, 2, 4, 8, 1, 1]	0.111	0.066	9.513
<Time>	[16, 16, 64, 2048, 8192, 128, 4, 16, 32, 1, 1]	0.268	0.029	44.02
<Area>	[2, 2, 16, 256, 512, 64, 2, 4, 16, 1, 1]	0.201	0.213	6.497
<Trade-off>	[4, 4, 128, 2048, 4096, 128, 2, 4, 4, 1, 1]	0.122	0.046	13.54
<>-efficient	walBerla			
	Design parameters [†]	Energy	Time	Area [‡]
<Energy>	[16, 16, 128, 2048, 8192, 128, 2, 4, 4, 1, 1]	0.537	0.056	42.29
<Time>	[64, 1, 128, 2048, 8192, 128, 2, 16, 32, 8, 8]	2.491	0.054	270.1
<Area>	[2, 2, 16, 256, 512, 64, 2, 8, 16, 1, 1]	1.206	1.251	6.497
<Trade-off>	[8, 8, 16, 2048, 8192, 128, 4, 8, 32, 1, 1]	0.576	0.092	24.36

*For STREAM, the core count is fixed to 16, since its application size depends on it.

[†]The parameters follow the same order as in TABLE I.

[‡]The objectives use the same units as in Fig 4.

[‡]The trade-off design, which considers all objectives as equally important, is obtained by minimizing the equally-weighted Chebyshev aggregation evaluated on multi-objective values.

optimization solutions based on surrogate modeling, for more significant results. Since BOOM-Explorer and Hexaly do not support parallel exploration, the total simulation budget is set to 1000 for all algorithms, including the model initialization budget of $n = 64$ to ensure a fair comparison. For others, we allow $n_{TR} = 4$ parallel simulations (which is also the number of trust regions) at the same time during exploration. Assuming the core count is the most influential design parameter here, we also modify BOOM-Explorer’s clustering parameter by the latter to enable it to be applied to our problem instances.

To compare exploration algorithms quantitatively, we propose three performance metrics: the hypervolume indicator, computed as the Lebesgue measure between the Pareto front and the nadir point of all Pareto fronts combined, the diversity indicator from Cai *et al.* [6] which evaluates the spreading and distribution of the front, and the normalized running time. In particular, the higher hypervolume, the better, and the lower diversity indicator and normalized running time, the better.

Fig. 4 provides an overview of the obtained Pareto fronts from SEBO and the baselines. The experimental results show that SEBO outperforms other baselines by around 1-7% in terms of hypervolume, indicating a higher overall quality of its Pareto front. Although the chosen baselines are top-performing DSE algorithms, SEBO still competes with them and demonstrates other strengths. For nearly the same hypervolume and normalized running time performances, SEBO provides a

much higher diversity of Pareto front than the commercial solution Hexaly by around 24% and than HyperMapper by 11%. Not only does SEBO return better Pareto-optimal designs related to their objectives values, but also provides a better diversification of solutions, which is essential for offering a broader range of efficient design choices to the architect. Furthermore, SEBO outperforms BOOM-Explorer by running $2\times$ faster and achieving a better Pareto front in the end. Interestingly, SEBO spent as much time as Hexaly to retrieve the Pareto front whereas the latter doesn’t support parallel simulations. The increasing computational cost induced by ensemble learning is exactly counterbalanced by the use of trust regions and parallelization methods, thereby confirming the observations made in previous sections.

In TABLE II, we present some key Pareto-optimal designs that may interest designers. First, the results seem to be coherent. If we observe area-efficient designs, these are the ones that aim to minimize the values of design parameters, thus proving the consistency of the chosen simulators in correctly modeling design behavior. Secondly, we show the trade-off-efficient designs, which correspond to the best designs for architects who treat all objectives as equally important. The trade-off designs offer particular characteristics and are not necessarily the “average” of extreme designs. For instance, the trade-off design for walBerla is around $2\times$ slower than the time-efficient one, but about $4\times$ and $11\times$ more energy and area-efficient respectively. The gains are not proportional. Lastly, increasing the size of the architecture beyond a certain point no longer leads to faster execution times. If we observe time-efficient designs, none of them have the maximum core count. Whereas manual exploration may miss these observations, automated DSE procedures, in contrast, can capture non-trivial interactions between design configurations and performance metrics, highlighting DSE algorithms as game changers for designing efficient architectures.

V. CONCLUSION AND PERSPECTIVES

In this work, we presented an automated framework based on our own DSE algorithm, SEBO, for efficient HPC processor design. With adapted features, SEBO returns qualitative and diversified Pareto designs with state-of-the-art performance. While we also provided key designs, selecting the optimal design according to the decision maker’s preferences remains difficult and deserves to be studied as the final step of DSE.

REFERENCES

- [1] C. Bai, Q. Sun, J. Zhai, Y. Ma, B. Yu, and M. D. Wong, "BOOM-Explorer: RISC-V BOOM Microarchitecture Design Space Exploration Framework," in *IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, 2021.
- [2] F. Bellard, "QEMU, a fast and portable dynamic translator," in *Proceedings of the annual conference on USENIX Annual Technical Conference (USENIX ATC)*, 2005.
- [3] G. Beltrame, L. Fossati, and D. Sciuto, "Decision-Theoretic Design Space Exploration of Multiprocessor Platforms," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2010.
- [4] T. Benoist, B. Estellon, F. Gardi, R. Megel, and K. Nouioua, "Local-Solver 1.x: a black-box local-search solver for 0-1 programming," *4OR*, 2011.
- [5] N. Binkert, B. Beckmann, G. Black, S. K. Reinhardt, A. Saidi, A. Basu, J. Hestness, D. R. Hower, T. Krishna, S. Sardashti, R. Sen, K. Sewell, M. Shoaib, N. Vaish, M. D. Hill, and D. A. Wood, "The gem5 simulator," *ACM SIGARCH Computer Architecture News*, 2011.
- [6] X. Cai, H. Sun, and Z. Fan, "A diversity indicator based on reference vectors for many-objective optimization," *Information Sciences*, 2018.
- [7] A. Charif, G. Busnot, R. Mameesh, T. Sassolas, and N. Ventroux, "Fast Virtual Prototyping for Embedded Computing Systems Design and Exploration," in *Proceedings of the 11th Workshop on Rapid Simulation and Performance Evaluation: Methods and Tools*, 2019.
- [8] L. T. Clark, V. Vashishtha, L. Shifren, A. Gujja, S. Sinha, B. Cline, C. Ramamurthy, and G. Yeric, "ASAP7: A 7-nm finFET predictive process design kit," *Microelectronics Journal*, 2016.
- [9] S. Das and S. Tesfamariam, "State-of-the-Art Review of Design of Experiments for Physics-Informed Deep Learning," *arXiv preprint*, 2022.
- [10] S. Daulton, M. Balandat, and E. Bakshy, "Differentiable Expected Hypervolume Improvement for Parallel Multi-Objective Bayesian Optimization," in *Proceedings of the 34th International Conference on Neural Information Processing Systems (NeurIPS)*, 2020.
- [11] S. Daulton, M. Balandat, and E. Bakshy, "Parallel Bayesian Optimization of Multiple Noisy Objectives with Expected Hypervolume Improvement," in *Proceedings of the 35th International Conference on Neural Information Processing Systems (NeurIPS)*, 2021.
- [12] S. Daulton, D. Eriksson, M. Balandat, and E. Bakshy, "Multi-objective Bayesian optimization over high-dimensional search spaces," in *Proceedings of the Thirty-Eighth Conference on Uncertainty in Artificial Intelligence (UAI)*, 2022.
- [13] A. Deshwal, S. Ament, M. Balandat, E. Bakshy, J. R. Doppa, and D. Eriksson, "Bayesian Optimization over High-Dimensional Combinatorial Spaces via Dictionary-based Embeddings," in *Proceedings of The 26th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2023.
- [14] J. J. Dongarra, J. Du Croz, S. Hammarling, and I. S. Duff, "A set of level 3 basic linear algebra subprograms," *ACM Transactions on Mathematical Software*, 1990.
- [15] K. Drezkowski, A. Grosnit, and H. Bou-Ammar, "Framework and benchmarks for combinatorial and mixed-variable bayesian optimization," in *Proceedings of the 37th International Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2023.
- [16] D. Eriksson, M. Pearce, J. Gardner, R. D. Turner, and M. Poloczek, "Scalable Global Optimization via Local Bayesian Optimization," in *Proceedings of the 33rd International Conference on Neural Information Processing Systems (NeurIPS)*, 2019.
- [17] C. Feichtinger, S. Donath, H. Köstler, J. Götz, and U. Rüde, "WaLBERla: HPC software design for computational engineering simulations," *Journal of Computational Science*, 2011.
- [18] P. I. Frazier, "Bayesian Optimization," in *INFORMS TutORials in Operations Research*, 2018.
- [19] T. Givargis, F. Vahid, and J. Henkel, "System-level exploration for Pareto-optimal configurations in parameterized systems-on-a-chip," in *IEEE/ACM International Conference on Computer Aided Design, IEEE/ACM Digest of Technical Papers*, 2001.
- [20] A. Grosnit, C. Malherbe, R. Tutunov, X. Wan, J. Wang, and H. B. Ammar, "BOiLS: Bayesian Optimisation for Logic Synthesis," in *Design, Automation Test in Europe Conference Exhibition (DATE)*, 2022.
- [21] S. Li, J. H. Ahn, R. D. Strong, J. B. Brockman, D. M. Tullsen, and N. P. Jouppi, "McPAT: an integrated power, area, and timing modeling framework for multicore and manycore architectures," in *Proceedings of the 42nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2009.
- [22] T.-R. Lin, D. Penney, M. Pedram, and L. Chen, "A Deep Reinforcement Learning Framework for Architectural Exploration: A Routerless NoC Case Study," in *IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2020.
- [23] J. McCalpin, "Memory bandwidth and machine balance in high performance computers," *IEEE Technical Committee on Computer Architecture Newsletter*, 1995.
- [24] N. Muralimanohar, R. Balasubramonian, and N. Jouppi, "Optimizing NUCA Organizations and Wiring Alternatives for Large Caches with CACTI 6.0," in *40th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2007.
- [25] L. Nardi, A. Souza, D. Koeplinger, and K. Olukotun, "HyperMapper: a Practical Design Space Exploration Framework," in *IEEE 27th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*, 2019.
- [26] M. Palesi and T. Givargis, "Multi-objective design space exploration using genetic algorithms," in *Proceedings of the Tenth International Symposium on Hardware/Software Codesign (CODES)*, 2002.
- [27] J. Panerati, D. Sciuto, and G. Beltrame, "Optimization Strategies in Design Space Exploration," in *Handbook of Hardware/Software Codesign*, 2017.
- [28] C. E. Rasmussen and C. K. I. Williams, *Gaussian Processes for Machine Learning*. The MIT Press, 2005.
- [29] B. C. Schafer and K. Wakabayashi, "Design Space Exploration Acceleration Through Operation Clustering," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2010.
- [30] B. Schölkopf and A. J. Smola, *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. The MIT Press, 2001.
- [31] R. Turner, D. Eriksson, M. McCourt, J. Kiili, E. Laaksonen, Z. Xu, and I. Guyon, "Bayesian Optimization is Superior to Random Search for Machine Learning Hyperparameter Tuning: Analysis of the Black-Box Optimization Challenge 2020," in *Proceedings of the NeurIPS 2020 Competition and Demonstration Track*, 2021.
- [32] S. Van den Steen, S. Eyerman, S. De Pestel, M. Mechri, T. E. Carlson, D. Black-Schaffer, E. Hagersten, and L. Eeckhout, "Analytical Processor Performance and Power Modeling Using Micro-Architecture Independent Characteristics," *IEEE Transactions on Computers*, 2016.
- [33] X. Wan, V. Nguyen, H. Ha, B. Ru, C. Lu, and M. A. Osborne, "Think Global and Act Local: Bayesian Optimisation over High-Dimensional Categorical and Mixed Search Spaces," in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021.
- [34] T. Wolf and M. Franklin, "Performance models for network processor design," *IEEE Transactions on Parallel and Distributed Systems*, 2006.
- [35] D. Wolpert and W. Macready, "No free lunch theorems for optimization," *IEEE Transactions on Evolutionary Computation*, 1997.
- [36] T.-T. Wong, W.-S. Luk, and P.-A. Heng, "Sampling with Hammersley and Halton Points," *Journal of Graphics Tools*, 1997.
- [37] K. Yu, J. Bi, and V. Tresp, "Active learning via transductive experimental design," in *Proceedings of the 23rd international conference on Machine learning (ICML)*, 2006.
- [38] J. Zhai, C. Bai, B. Zhu, Y. Cai, Q. Zhou, and B. Yu, "McPAT-Calib: A Microarchitecture Power Modeling Framework for Modern CPUs," in *2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, 2021.